

Assignment 3 Optimization Problems

Shuifan Wang `shepherd.wang@foxmail.com`

April 4, 2025

Problem 1

The Minimum of Schaffer Function.

$$f(x, y) = 0.5 + \frac{\sin^2 \sqrt{x^2 + y^2} - 0.5}{(1 + 0.001(x^2 + y^2))^2}$$

The result obtained using the genetic algorithm is not very satisfactory, with the minimum value being trapped in a local convergence at $(x, y) = (-2.3724, -2.0547)$, $f(x, y) = 0.009716$. However, using the multi-start `fminunc` directly yielded the correct result $(x, y) = (0, 0)$, $f(x, y) = 0$, as the starting points happened to include the location of the global minimum.

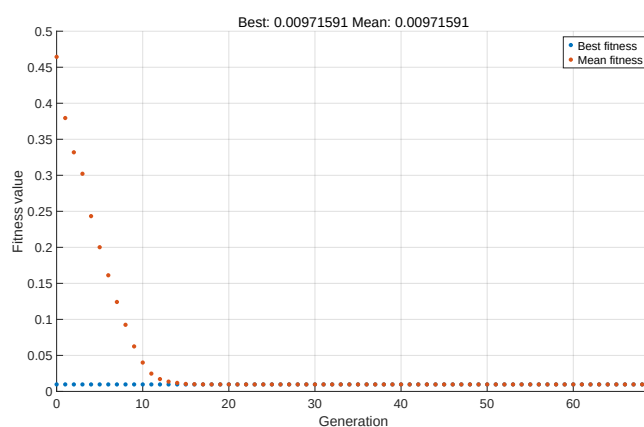


Figure 1: Problem 1

```

1  %-----Problem 1-----
2  diary('output1.txt');
3  diary on;
4
5  clc; clear; close all;
6
7  % First-Order Optimality Conditions for Trust-Region Algorithm
8
9  x0_list = [10, 10; -10, -10; 0, 0; -10, 10; 10, -10];
10 schaffer = @(x) 0.5 + ((sin(sqrt(x(1)^2 + x(2)^2)))^2 - 0.5)/(1 + ...
    0.001*(x(1)^2 + x(2)^2))^2;
11 options = optimset;
12 options.TolFun = 1.0e-10;
13 options.Tolx = 1.0e-10;
14 options = optimset('PlotFcns', @optimplotfirstorderopt);
15
16 best_FVAL = Inf;
17 best_X = [];
18 for i = 1:size(x0_list, 1)
19     x0 = x0_list(i, :);
20     [X, FVAL] = fminunc(schaffer, x0, options);
21     if FVAL < best_FVAL
22         best_FVAL = FVAL;
23         best_X = X;
24     end
25 end
26 fprintf('Optimal Solution: x = %.4f, y = %.4f, f(x,y) = %.6f\n', ...
    best_X(1), best_X(2), best_FVAL);
27
28 clc; clear; close all;
29
30 % Genetic Algorithm
31
32 hybrid_options = optimoptions('fmincon', 'Algorithm', 'sqp');
33 options = optimoptions('ga', ...
34     'MaxGenerations', 5000, ...
35     'PopulationSize', 1000, ...
36     'CrossoverFraction', 0.8, ...
37     'MutationFcn', @mutationadaptfeasible, ...
38     'FunctionTolerance', 1e-12, ...
39     'HybridFcn', {@fmincon, hybrid_options}, ...
40     'PlotFcn', @gaplotbestf);
41 lb = [-10, -10];
42 ub = [10, 10];
43 schaffer = @(x) 0.5 + ((sin(sqrt(x(1)^2 + x(2)^2)))^2 - 0.5)/(1 + ...
    0.001*(x(1)^2 + x(2)^2))^2;
44
45 [X, FVAL] = ga(schaffer, 2, [], [], [], [], lb, ub, [], options);
46
47 fprintf('Optimal Solution: x = %.4f, y = %.4f, f(x,y) = %.6f\n', ...
    X(1), X(2), FVAL);
48 set(gca, 'FontSize', 18);
49 set(gcf, 'Position', [100, 100, 1600, 900]);
50 exportgraphics(gcf, 'Problem1_ga.pdf', 'ContentType', 'vector');
51
52 diary off;

```

Problem 2

$$\begin{aligned} \min f(x) &= 10x_1^3 + x_1x_2^2 + x_3(x_1^2 + x_2^2) \\ \text{s.t. } &\begin{cases} \sqrt{x_1^2 + x_2^2} - x_3 - 10 \leq 0 \\ \sqrt{x_1^2 + x_2^2} + x_3 - 3 \leq 0 \end{cases} \end{aligned}$$

The results obtained using **fmincon** or simulated annealing algorithm (with penalty function method) is consistent with the result calculated theoretically: $(x_1, x_2, x_3) = (-6.5, 0, -3.5)$, $f = -2894.124996$. While the result of pattern search algorithm is $(x_1, x_2, x_3) = (-3.3166, -5.0000, -4.0000)$, $f = -591.745018$, far from satisfactory. It also indicates that the start point might influence. Moreover, simulated annealing converges with a slow trend after 1000 times of iteration. Therefore, it is advisable to first use simulated annealing algorithm to determine the start point and then use pattern search algorithm.

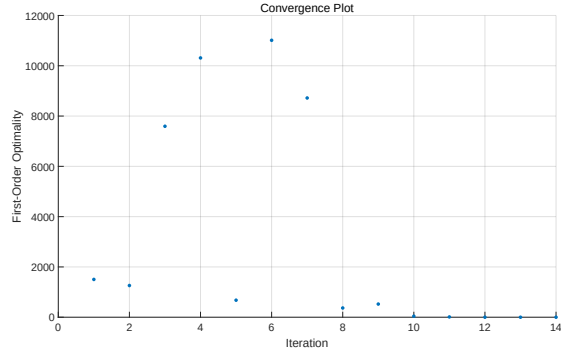


Figure 2: Problem 2tr

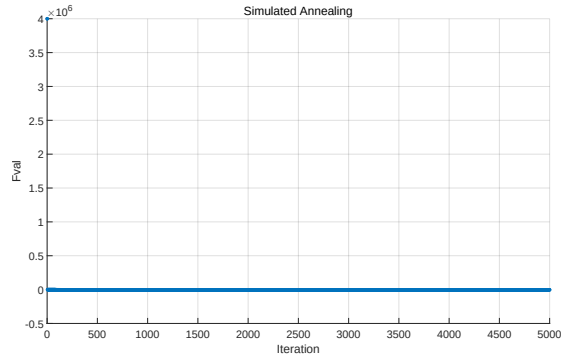


Figure 3: Problem 2sa

```

1  %-----Problem 2-----
2  diary('output2.txt');
3  diary on;
4
5  clc; clear; close all;
6
7  % First-Order Optimality Conditions for Trust-Region Algorithm
8
9  f = @(x) 10*x(1)^3 + x(1)*x(2)^2 + x(3)*(x(1)^2 + x(2)^2);
10 nonlcon = @(x) deal([ sqrt(x(1)^2 + x(2)^2) - x(3) - 10;
11                    sqrt(x(1)^2 + x(2)^2) + x(3) - 3 ], []);
12 x0 = [0, -5, 0];
13 options = optimset;
14 options.TolFun = 1.0e-10;
15 options.Tolx = 1.0e-10;
16 options = optimset('PlotFcns', @optimplotfirstorderopt);
17
18 [X, FVAL, EXITFLAG, OUTPUT] = fmincon(f, x0, [], [], [], [], [], ...
19                                     [], nonlcon, options);
20 fprintf('Optimal Solution: x1 = %.4f, x2 = %.4f, x3 = %.4f, f = ...
21         %.6f\n', X(1), X(2), X(3), FVAL);
22 title('Convergence Plot');
23 xlabel('Iteration');
24 ylabel('First-Order Optimality');
25 set(gca, 'FontSize', 18);
26 set(gcf, 'Position', [100, 100, 1600, 900]);
27 exportgraphics(gcf, 'Problem1_tr.pdf', 'ContentType', 'vector');
28
29 % Pattern Search Algorithm
30
31 f = @(x) 10*x(1)^3 + x(1)*x(2)^2 + x(3)*(x(1)^2 + x(2)^2);
32 nonlcon = @(x) deal([ sqrt(x(1)^2 + x(2)^2) - x(3) - 10;
33                    sqrt(x(1)^2 + x(2)^2) + x(3) - 3 ], []);
34 options = optimoptions('patternsearch', ...
35                        'Display', 'iter', ...
36                        'MaxIterations', 500);
37 x0 = [0, -5, 0];
38
39 [X, FVAL] = patternsearch(f, x0, [], [], [], [], [], [], nonlcon, ...
40                          options);
41
42 fprintf('Optimal Solution: x1 = %.4f, x2 = %.4f, x3 = %.4f, f = ...
43         %.6f\n', X(1), X(2), X(3), FVAL);
44
45 % Simulated Annealing Algorithm with Penalty Function Method
46
47 clc; clear; close all;
48
49 f_original = @(x) 10*x(1)^3 + x(1)*x(2)^2 + x(3)*(x(1)^2 + x(2)^2);
50 nonlcon = @(x) [ sqrt(x(1)^2 + x(2)^2) - x(3) - 10;
51                sqrt(x(1)^2 + x(2)^2) + x(3) - 3 ];
52 P = 1e6;
53 f = @(x) f_original(x) + P * sum(max(nonlcon(x), 0).^2);
54 lb = [-20, -20, -20];
55 ub = [20, 20, 20];
56 options = optimoptions('simulannealbnd', ...

```

```

54     'MaxIterations', 280, ...
55     'FunctionTolerance', 1e-12, ...
56     'PlotFcn', @splotbestf);
57
58 [X, FVAL] = simulannealbnd(f, [0, -5, 0], lb, ub, options);
59
60 fprintf('Optimal Solution: x1 = %.4f, x2 = %.4f, x3 = %.4f, f = ...
        %.6f\n', X(1), X(2), X(3), FVAL);
61 title('Simulated Annealing');
62 xlabel('Iteration');
63 ylabel('Fval');
64 set(gca, 'FontSize', 18);
65 set(gcf, 'Position', [100, 100, 1600, 900]);
66 exportgraphics(gcf, 'Problem1_sa.pdf', 'ContentType', 'vector');
67
68 diary off;

```

Problem 3

x (AU)	y (AU)
5.764	0.648
6.286	1.202
6.759	1.823
7.168	2.526
7.480	3.360

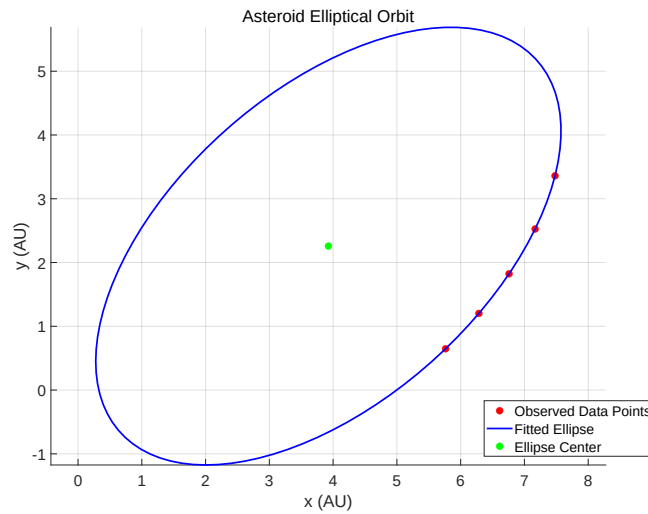


Figure 4: Problem 3

Elliptical Orbit: $0.1690x^2 - 0.1892xy + 0.1910y^2 - 0.8996x - 0.1191y + 0.2751 = 0$.

Ellipse Center: $(x_0, y_0) = (3.9257, 2.2569)$.

Semi-Major Axis: $b = 4.3799$, Semi-Minor Axis: $a = 2.4299$.

```

1  %-----Problem 3-----
2  diary('output3.txt');
3  diary on;
4
5  clc; clear; close all;
6
7  x = [5.764, 6.286, 6.759, 7.168, 7.480]';
8  y = [0.648, 1.202, 1.823, 2.526, 3.360]';
9  A = [x.^2, x.*y, y.^2, x, y, ones(size(x))];
10
11 % Solving the Least Squares Problem Using Singular Value ...
12 % Decomposition
13 [U, S, V] = svd(A);
14 P = V(:, end);
15 A = P(1); B = P(2); C = P(3);
16 D = P(4); E = P(5); F = P(6);
17 x0 = (2 * C * D - B * E) / (B^2 - 4 * A * C);
18 y0 = (2 * A * E - B * D) / (B^2 - 4 * A * C);
19
20 % Calculate the Semi-Major Axis a and the Semi-Minor Axis b
21 theta = 0.5 * atan2(B, A - C);
22 number = 2 * (A * x0^2 + C * y0^2 + B * x0 * y0 - F);
23 denom1 = A + C + sqrt((A - C)^2 + B^2);
24 denom2 = A + C - sqrt((A - C)^2 + B^2);
25 a = sqrt(number / denom1);
26 b = sqrt(number / denom2);
27
28 % Plot the Orbit
29 theta_grid = linspace(0, 2*pi, 100);
30 X_ellipse = a * cos(theta_grid);
31 Y_ellipse = b * sin(theta_grid);
32 R = [cos(theta), -sin(theta); sin(theta), cos(theta)];
33 ellipse_rotated = R * [X_ellipse; Y_ellipse];
34 X_ellipse = ellipse_rotated(1, :) + x0;
35 Y_ellipse = ellipse_rotated(2, :) + y0;
36
37 figure; hold on; axis equal;
38 plot(x, y, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 8);
39 plot(X_ellipse, Y_ellipse, 'b-', 'LineWidth', 2);
40 plot(x0, y0, 'go', 'MarkerFaceColor', 'g', 'MarkerSize', 8);
41 legend('Observed Data Points', 'Fitted Ellipse', 'Ellipse ...
42 % Center', 'Location', 'Best');
43 xlabel('x (AU)');
44 ylabel('y (AU)');
45 title('Asteroid Elliptical Orbit');
46 grid on;
47
48 % Save and Print
49 set(gcf, 'FontSize', 18);
50 set(gcf, 'Position', [100, 100, 1600, 900]);
51 exportgraphics(gcf, 'asteroid_orbit.pdf', 'ContentType', 'vector');

```

```

51 fprintf('Elliptical Orbit: %.4fx^2 + %.4fxy + %.4fy^2 + %.4fx + ...
52         %.4fy + %.4f = 0\n', A, B, C, D, E, F);
53 fprintf('Ellipse Center: (%.4f, %.4f)\n', x0, y0);
54 fprintf('Semi-Major Axis: %.4f, Semi-Minor Axis: %.4f\n', a, b);
55
56 diary off;
57
58 % Output:
59 % Elliptical Orbit: 0.1690x^2 + -0.1892xy + 0.1910y^2 + -0.8996x ...
60         + -0.1191y + 0.2751 = 0
61 % Ellipse Center: (3.9257, 2.2569)
62 % Semi-Major Axis: 2.4299, Semi-Minor Axis: 4.3799

```

Problem 4

$$\min f(x) = -c_1 e^{-0.2\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}} - e^{\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i)} + c_1 + e, \quad c_1 = 20, \quad n = 1, 2, 10.$$

For all n , $f(x)_{\min} = 0$ and $x = 0$. As for $n = 10$, prior trials have shown that the optimal results often converge to a local optimum. Therefore, we choose to dynamically adjust the cognitive parameter and social parameter, and increase the number of particles and iterations (even though the convergence is good). We increase `max_iter` to 500 and `n_particles` to 10000. In early stage, we increase `c1` to let particles explore more possible solutions. In late stage, we enlarge `c2` to let particles focus on the optimal solution, improving the convergence speed. [For the video demonstration at \$n = 1\$, click here.](#)

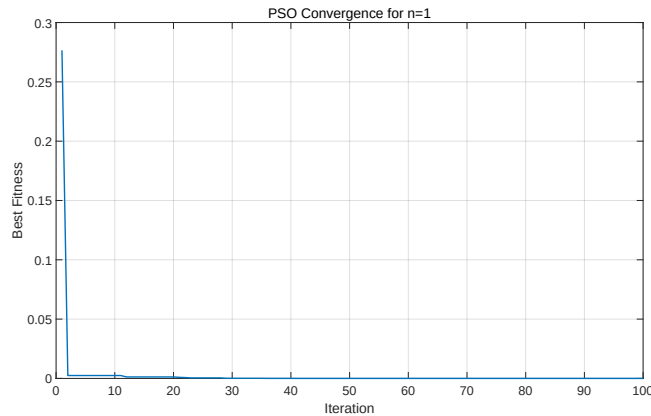


Figure 5: Problem 4, $n = 1$

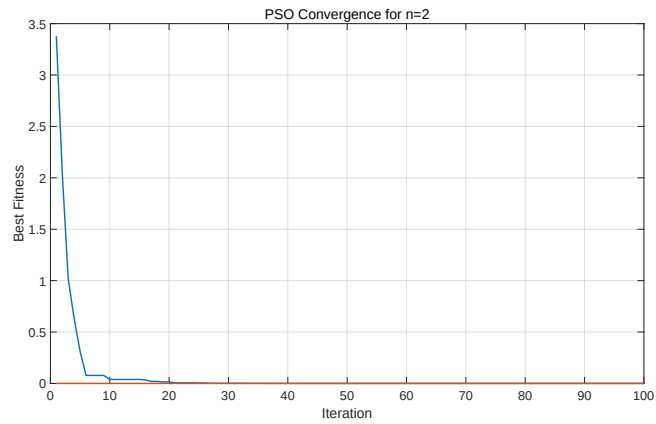


Figure 6: Problem 4, $n = 2$

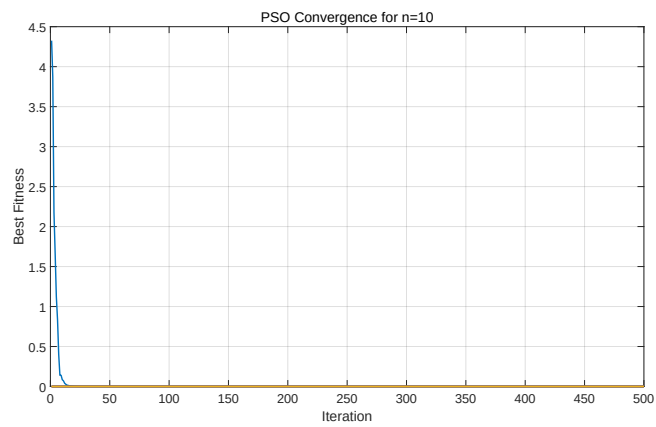


Figure 7: Problem 4, $n = 10$

```

1  %-----Problem 4-----
2  warning('off', 'all');
3  diary('output4.txt');
4  diary on;
5
6  clc; clear; close all;
7
8  n = [1, 2, 10];
9  n_particles = 100;
10 max_iter = 100;
11 w = 0.7;           % Inertia Weight
12 c1 = 1.5;          % Cognitive Parameter
13 c2 = 1.5;          % Social Parameter
14 bounds = [-5, 5]; % Search Range

```

```

15
16 % Ackley function
17 ackley = @(x) -20 * exp(-0.2 * sqrt(sum(x.^2) / length(x))) ...
18             - exp(sum(cos(2*pi * x)) / length(x)) + 20 + exp(1);
19
20 for i = n
21     fprintf('Running PSO for n = %d\n', i);
22
23     if i == 10
24         % More Particles and Iterations
25         n_particles = 10000;
26         max_iter = 500;
27     end
28
29 % Initialize particles
30 positions = bounds(1) + (bounds(2) - bounds(1)) * ...
31             rand(n_particles, i);
32 velocities = zeros(n_particles, i);
33 pbest = positions;
34 gbest = positions(1, :);
35 pbest_fitness = zeros(n_particles, 1);
36 for j = 1:n_particles
37     pbest_fitness(j) = ackley(pbest(j, :));
38 end
39 gbest_fitness = ackley(gbest);
40 history = zeros(max_iter, i);
41
42 % Initialize a Video Object
43 if i == 1
44     video_filename = 'pso_optimization';
45     v = VideoWriter(video_filename, 'MPEG-4');
46     v.FrameRate = 10;
47     open(v);
48 end
49
50 for iter = 1:max_iter
51     if i == 10
52         % Early Stage (large c1): Particles Explore More ...
53         % Possible Solutions.
54         c1 = 2.5 - iter * (2.5 - 0.5) / max_iter;
55         % Late Stage (large c2): Particles Focus on the ...
56         % Optimal Solution, Improving the Convergence Speed.
57         c2 = 0.5 + iter * (2.5 - 0.5) / max_iter;
58         w = 0.4;
59     end
60
61     for j = 1:n_particles
62         p = positions(j, :);
63         fitness = ackley(p);
64
65         % Update Personal Best
66         if fitness < pbest_fitness(j)
67             pbest(j, :) = p;
68             pbest_fitness(j) = fitness;
69         end
70
71         % Update Global Best

```

```

69         if fitness < gbest_fitness
70             gbest = p;
71             gbest_fitness = fitness;
72         end
73     end
74
75     for j = 1:n_particles
76         r1 = rand();
77         r2 = rand();
78
79         % Update Velocity
80         velocities(j, :) = w * velocities(j, :) + ...
81             c1 * r1 * (pbest(j, :) - ...
82                 positions(j, :)) + ...
83             c2 * r2 * (gbest - positions(j, :));
84
85         % Update Position
86         new_position = positions(j, :) + velocities(j, :);
87
88         % Boundary handling
89         new_position = max(min(new_position, bounds(2)), ...
90             bounds(1));
91
92         positions(j, :) = new_position;
93     end
94
95     history(iter) = gbest_fitness;
96
97     % Visualization
98     if i == 1
99         clf;
100         fplot(ackley, bounds, 'k-', 'LineWidth', 2);
101         hold on;
102         scatter(positions, arrayfun(ackley, positions), 'ro', ...
103             'filled');
104         scatter(gbest, ackley(gbest), 'b*', 'LineWidth', 2);
105         title(sprintf('Iteration %d, Best Value: %.4f', iter, ...
106             gbest_fitness));
107         xlabel('x');
108         ylabel('f(x)');
109         frame = getframe(gcf);
110         writeVideo(v, frame);
111         pause(0.05);
112     end
113 end
114
115 if i == 1
116     close(v);
117     fprintf('Video saved as %s\n', video_filename);
118 end
119
120 % Print the Final Result
121 fprintf('Best solution found for n=%d: %.6f\n', i, ...
122     gbest_fitness);
123
124 % Plot Convergence Curve
125 figure;

```

```

121     plot(history, 'LineWidth', 2);
122     xlabel('Iteration');
123     ylabel('Best Fitness');
124     title(sprintf('PSO Convergence for n=%d', i));
125     grid on;
126
127     set(gca, 'FontSize', 18);
128     set(gcf, 'Position', [100, 100, 1600, 900]);
129     exportgraphics(gcf, sprintf('Problem4_n%d.pdf', i), ...
        'ContentType', 'vector');
130 end
131
132 diary off;
133
134 % Output:
135 % Running PSO for n = 1
136 % Video saved as pso_optimization.avi
137 % Best solution found for n=1: 0.000000
138 % Running PSO for n = 2
139 % Best solution found for n=2: 0.000000
140 % Running PSO for n = 10
141 % Best solution found for n=10: 0.000000

```